



National University of Computer and Emerging Sciences

Laboratory Manuals

for

Computer Networks - Lab

(CL -3001)

Department of Computer Science

FAST-NU, Lahore, Pakistan

Instructor:

Muhammad Nouman Hanif

Lab Manual 03

Objective:

- Introduction to Wireshark
- Analyzing HTTP, DNS, and TCP packets using Wireshark

Introduction to Wireshark

Wireshark is a free and open-source packet analyzer. It is used for network troubleshooting, analysis, software and communications protocol development, and education. Wireshark, a network analysis tool formerly known as Ethereal, captures packets in real time and displays them in human-readable format. Wireshark includes filters, color-coding and other features that let you dig deep into network traffic and inspect individual packets.

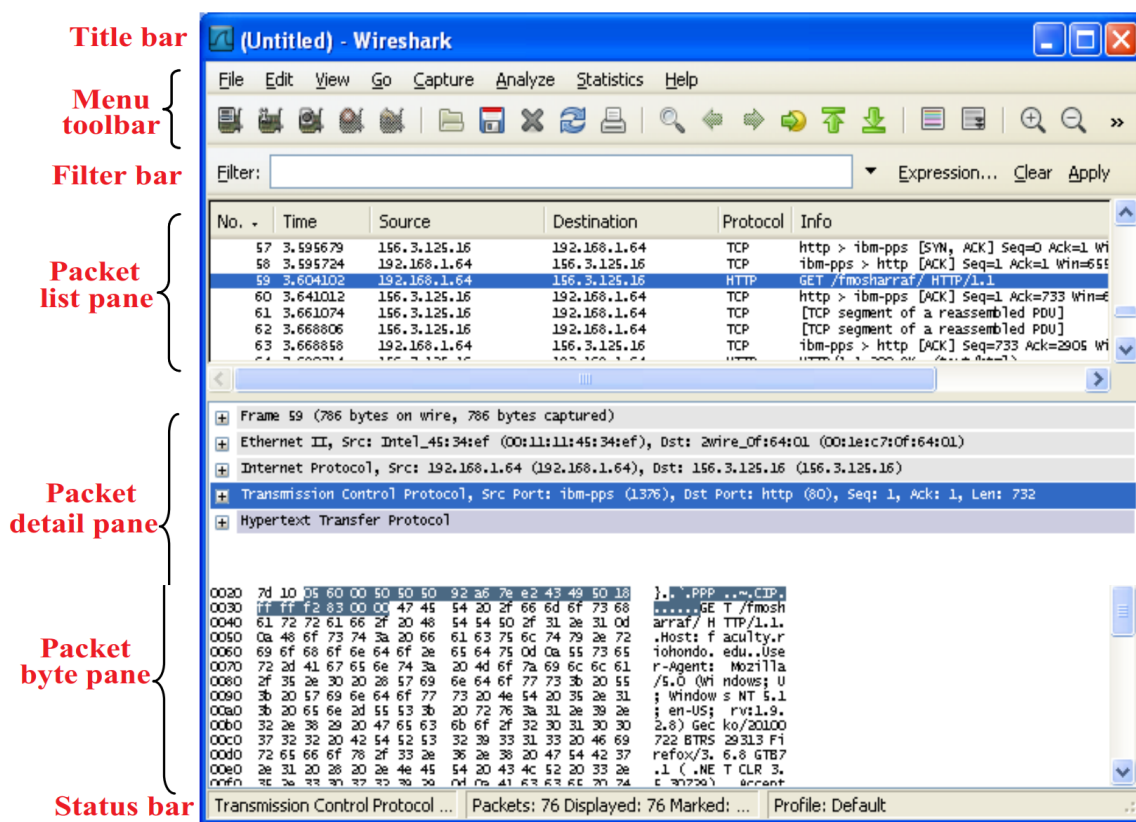


Figure 1: Wireshark Graphical User Interface

Why we use Wireshark?

Wireshark has many uses, including troubleshooting networks that have performance issues. Cybersecurity professionals often use Wireshark to trace connections, view the contents of suspect network transactions and identify bursts of network traffic.

Background: HTTP vs. HTTPS

Hypertext Transfer Protocol (HTTP)

Hypertext Transfer Protocol (HTTP) is an application layer protocol used in networking. When you type any web address in your web browser, your browser acts as a client, and the computer with the requested information acts as a server. The client uses the HTTP protocol to request information from the server. After the request is completed, the server responds back to the client, typically in the form of a web page.

Hypertext Transfer Protocol Secure (HTTPS)

Hypertext Transfer Protocol Secure (HTTPS) is a more secure way to access the web, combining HTTP with the SSL/TLS protocol. Communication is encrypted, meaning no one else can know what you are looking for. This is essential for websites where security is required, such as banking websites, payment gateways, and emails. For an HTTPS connection, the server requires a trusted and signed public key certificate.

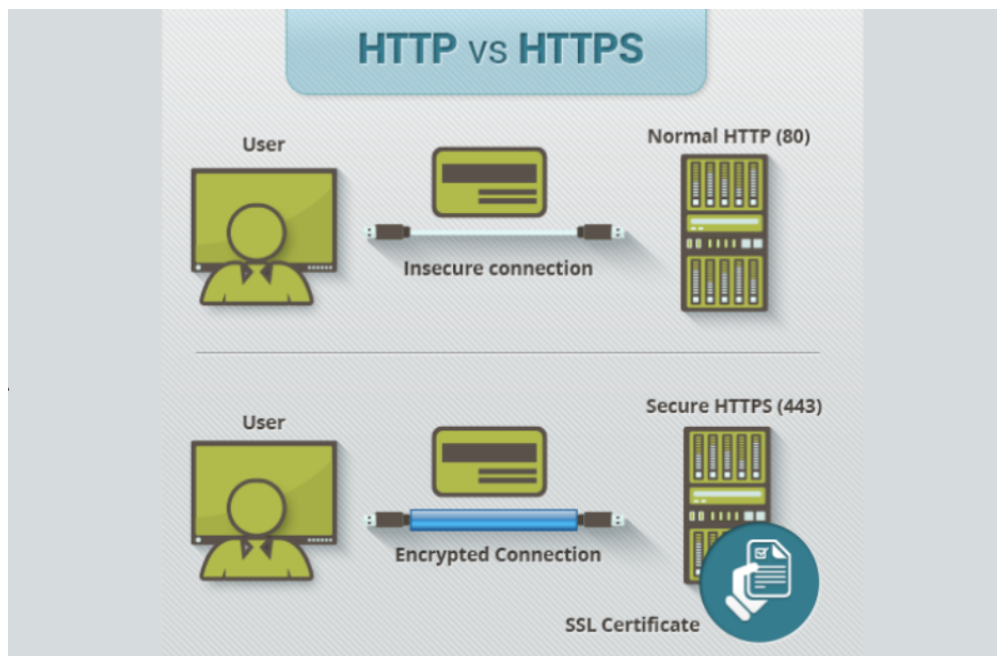


Figure 2: Visual difference between an insecure HTTP and a secure HTTPS connection.

Difference between HTTP & HTTPS

The table below highlights the major differences between the two protocols.

HTTP	HTTPS
URL begins with "http://"	URL begins with "https://"
It uses port 80 for communication.	It uses port 443 for communication.
Unsecured.	Secured.
Operates at Application Layer.	Operates at Transport Layer.
No encryption.	Encryption is present.

In-Lab Tasks: HTTP Analysis

1. The Basic HTTP GET/Response Interaction

Let's begin by downloading a simple HTML file. Do the following:

1. Start your browser and Wireshark.
2. In Wireshark's display filter, enter `http` so that only HTTP messages will be displayed.
3. Begin Wireshark packet capture.
4. In your browser, enter the URL: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html>
5. Your browser should display a simple one-line HTML file.
6. Stop Wireshark packet capture.

Your Wireshark window should look similar to the one shown below. If you cannot run Wireshark

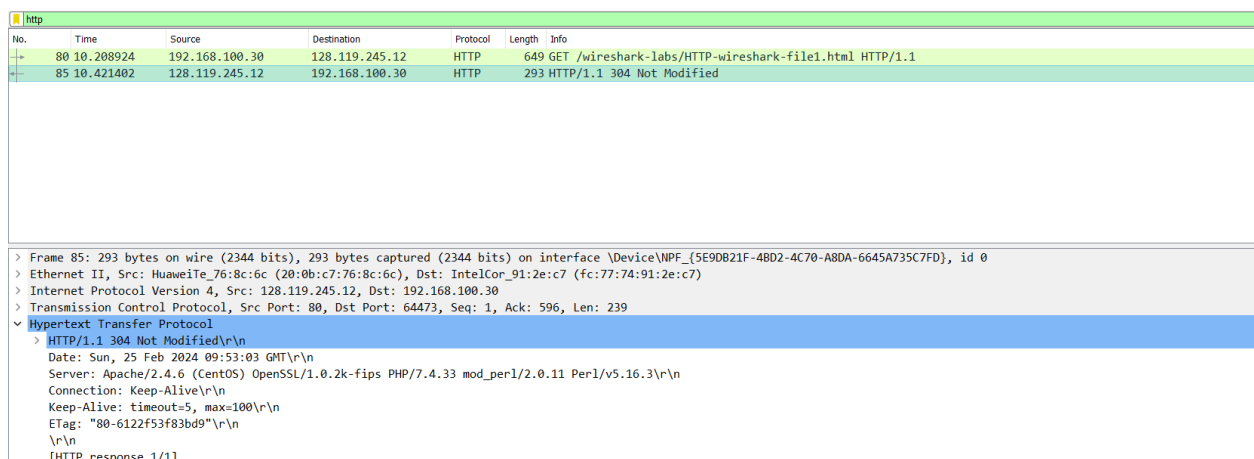


Figure 3: Wireshark capture of a basic HTTP GET and a "304 Not Modified" response.

on a live network, you can use the `http-ethereal-trace-1` file from the provided 'wireshark-traces.zip' file. By looking at the captured GET and response messages, answer the following questions:

1. Is your browser running HTTP version 1.0 or 1.1? What version of HTTP is the server running?
2. What languages (if any) does your browser indicate that it can accept to the server?
3. What is the IP address of your computer and the 'gaia.cs.umass.edu' server?
4. What is the status code returned from the server to your browser?
5. When was the HTML file that you are retrieving last modified at the server?
6. How many bytes of content are being returned to your browser?

2. The HTTP Conditional GET/Response Interaction

Most web browsers perform object caching and use a conditional GET. Before starting, empty your browser's cache.

1. Start your web browser with a cleared cache.
2. Start Wireshark packet capture.
3. Enter the URL: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>
4. Quickly enter the same URL again (or press refresh).
5. Stop packet capture and filter for **http**.

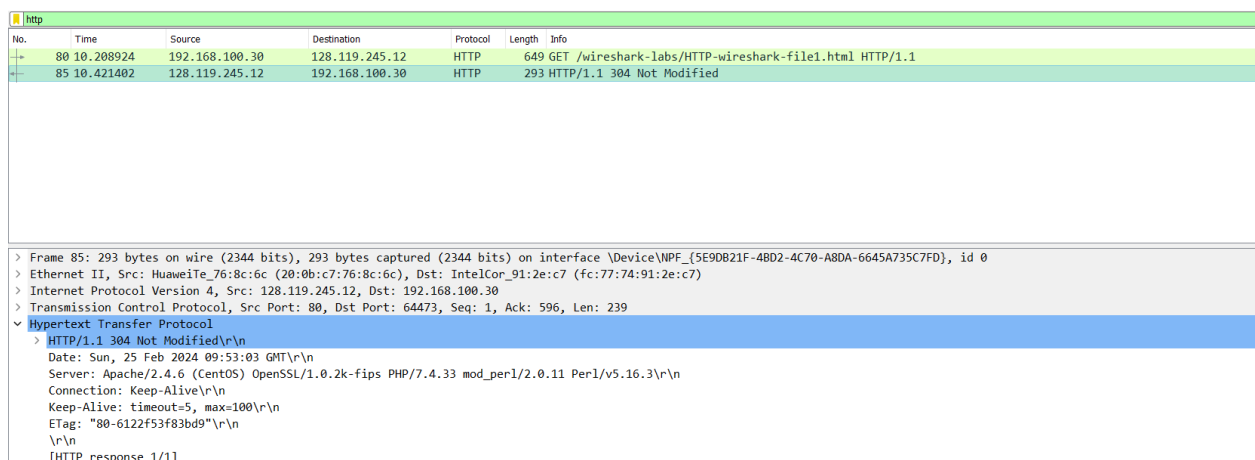


Figure 4: Wireshark capture showing a conditional GET and "304 Not Modified" response.

If you cannot run a live capture, use the **http-ethereal-trace-2** file. Answer the following questions:

1. Inspect the first HTTP GET request. Do you see an "IF-MODIFIED-SINCE" line?
2. Inspect the server response. Did the server explicitly return the contents of the file? How can you tell?
3. Inspect the second HTTP GET request. Do you see an "IF-MODIFIED-SINCE" line? What information follows it?
4. What is the HTTP status code in response to the second GET? Did the server return the file's contents? Explain.

3. Retrieving Long Documents

Now let's download a long HTML file.

1. Start your browser with a cleared cache and start Wireshark capture.
2. Enter the URL: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html>

3. Your browser will display the US Bill of Rights.
4. Stop capture and filter for `http`.

If you cannot run a live capture, use the `http-ethereal-trace-3` file. The single HTTP response will be broken into several TCP segments.

1. How many HTTP GET request messages did your browser send? Which packet number contains the GET?
2. Which packet number contains the status code for the response? What is the status code?
3. How many data-containing TCP segments were needed to carry the single HTTP response?

4. HTML Documents with Embedded Objects

This exercise explores what happens when a browser downloads a file with embedded objects.

1. Start your browser with a cleared cache and start Wireshark capture.
2. Enter the URL: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html>
3. Your browser will display a short HTML file with two images.
4. Stop capture and filter for `http`.

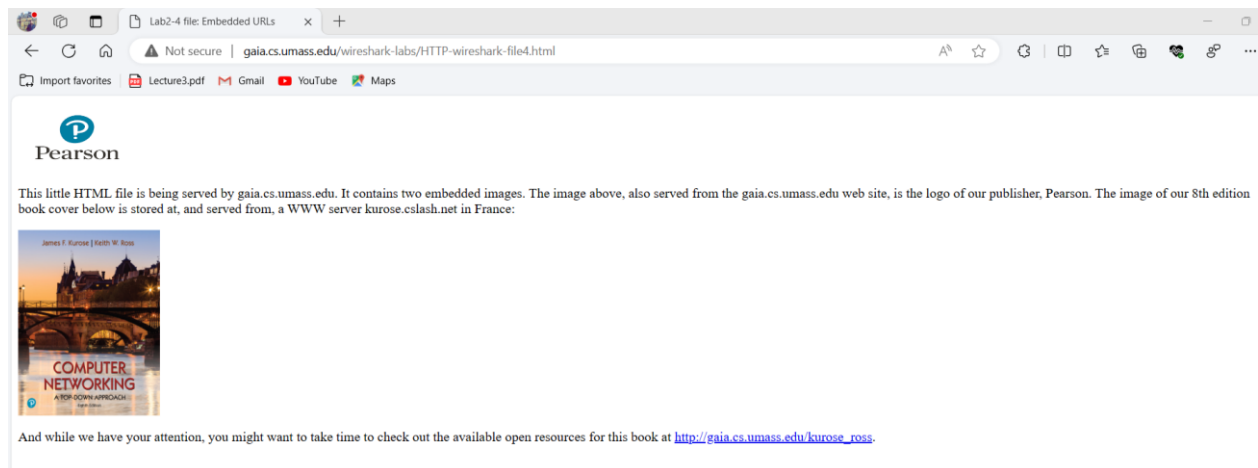


Figure 5: A web page with embedded objects from different servers.

If unable to perform a live capture, use the `http-ethereal-trace-4` file.

1. How many HTTP GET request messages did your browser send? To which Internet addresses were they sent?
2. Can you tell whether your browser downloaded the two images serially, or in parallel? Explain.

5. HTTP Authentication

Let's visit a password-protected website.

1. Start your browser with a cleared cache and start Wireshark capture.

2. Enter the URL: http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html
3. In the pop-up box, enter the username **wireshark-students** and password **network**.
4. Stop capture and filter for **http**.

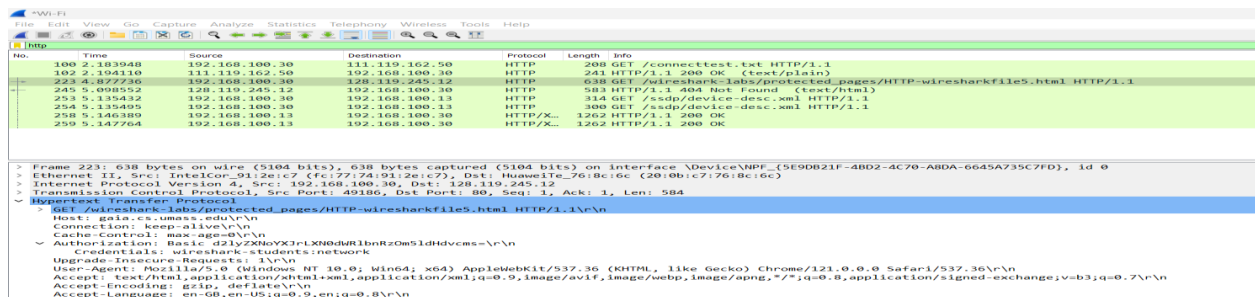


Figure 6: Wireshark capture showing the Authorization header field in an HTTP GET.

If unable to perform a live capture, use the **http-ethereal-trace-5** file.

1. What is the server's response (status code and phrase) to the initial HTTP GET message?
2. When your browser sends the HTTP GET message for the second time, what new field is included?

In-Lab Tasks: DNS Analysis

The Domain Name System (DNS) translates hostnames to IP addresses. We'll explore the client side of DNS using 'nslookup' and 'ipconfig'.

1. nslookup

The 'nslookup' tool allows a host to query any DNS server for a DNS record.

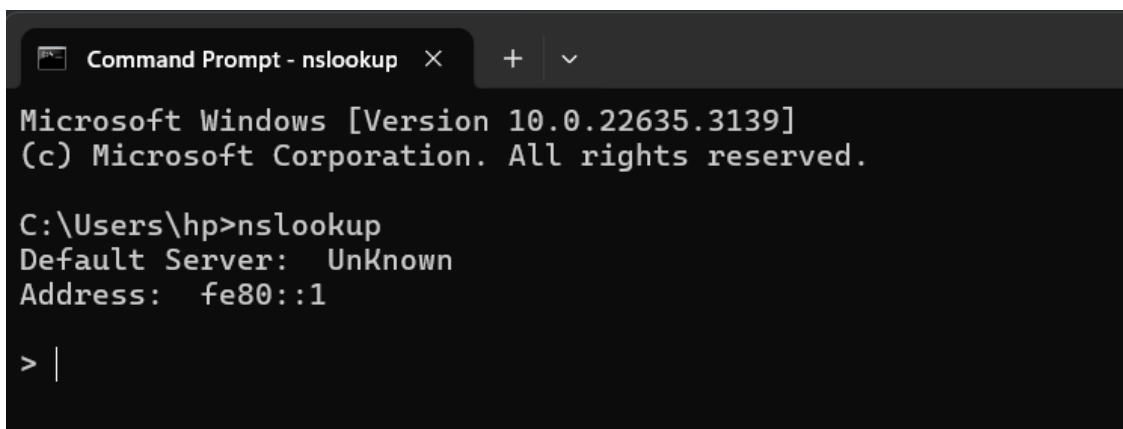


Figure 7: Example of using nslookup to query DNS records.

1. Run 'nslookup' to obtain the IP address of a web server in Asia. What is its IP address?

2. Run 'nslookup' to determine the authoritative DNS servers for a university in Europe.
3. Run 'nslookup' so that one of the DNS servers from the previous question is queried for the mail servers for Yahoo! mail.

2. ipconfig

'ipconfig' (Windows) and 'ifconfig' (Linux/Unix) show TCP/IP information. You can view all configuration details with `ipconfig /all`, display the local DNS cache with `ipconfig /displaydns`, and clear the cache with `ipconfig /flushdns`.

3. Tracing DNS with Wireshark

Let's capture DNS packets generated by web surfing.

1. Use `ipconfig /flushdns` to empty your DNS cache.
2. Empty your browser's cache.
3. Start Wireshark capture.
4. With your browser, visit <http://www.ietf.org>.
5. Stop packet capture.

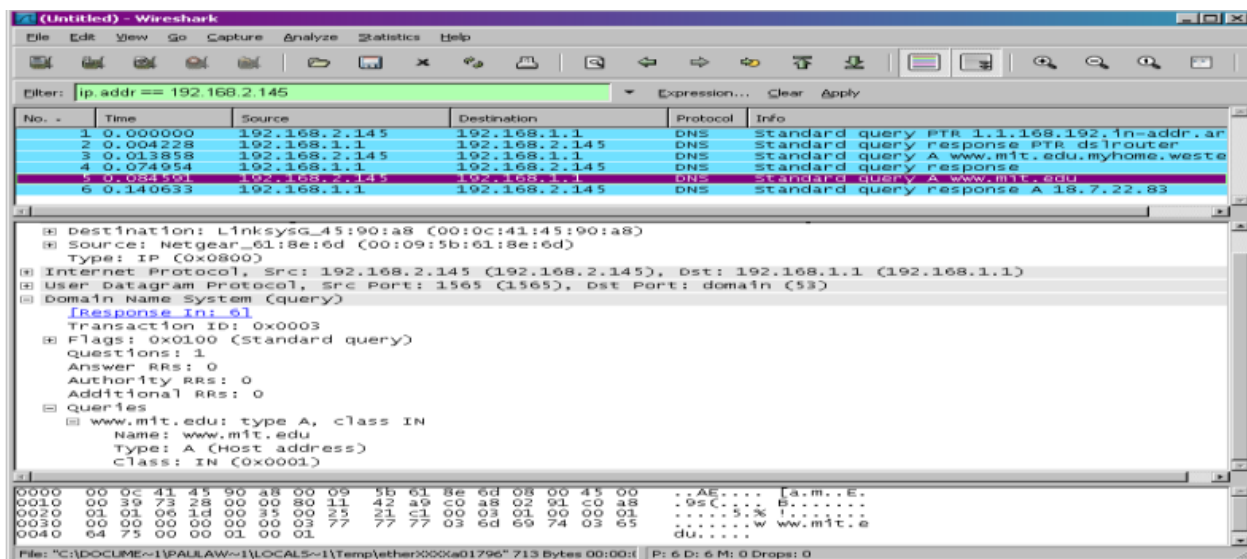


Figure 8: A Wireshark capture showing DNS query and response packets.

Answer the following questions:

1. Locate the DNS query and response messages. Are they sent over UDP or TCP?
2. What is the destination port for the DNS query message? What is the source port of the DNS response message?
3. To what IP address is the DNS query message sent? Is this the same as your local DNS server's IP address (found using 'ipconfig')?

4. Examine the DNS query message. What "Type" of DNS query is it? Does it contain any "answers"?
5. Examine the DNS response message. How many "answers" are provided? What does each answer contain?

In-Lab Tasks: TCP Analysis

1. Capturing a Bulk TCP transfer

We will capture the TCP transfer of a large file from your computer to a remote server using an HTTP POST.

1. Download the text file <http://gaia.cs.umass.edu/wireshark-labs/alice.txt> and save it to your computer.
2. Go to the upload page: <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>.
3. Use the "Browse" button to select the 'alice.txt' file you just saved.
4. Start Wireshark packet capture.
5. Return to your browser and press the "Upload alice.txt file" button.
6. Wait for the confirmation message, then stop Wireshark capture.

2. A First Look at The Captured Trace

Filter the captured packets by entering `tcp` in the display filter. You should see the three-way handshake, the HTTP POST message, and the corresponding TCP ACKs.

1. What is the IP address and TCP port number used by the client computer transferring the file?
2. What is the IP address of 'gaia.cs.umass.edu'? On what port number is it sending and receiving TCP segments for this connection?

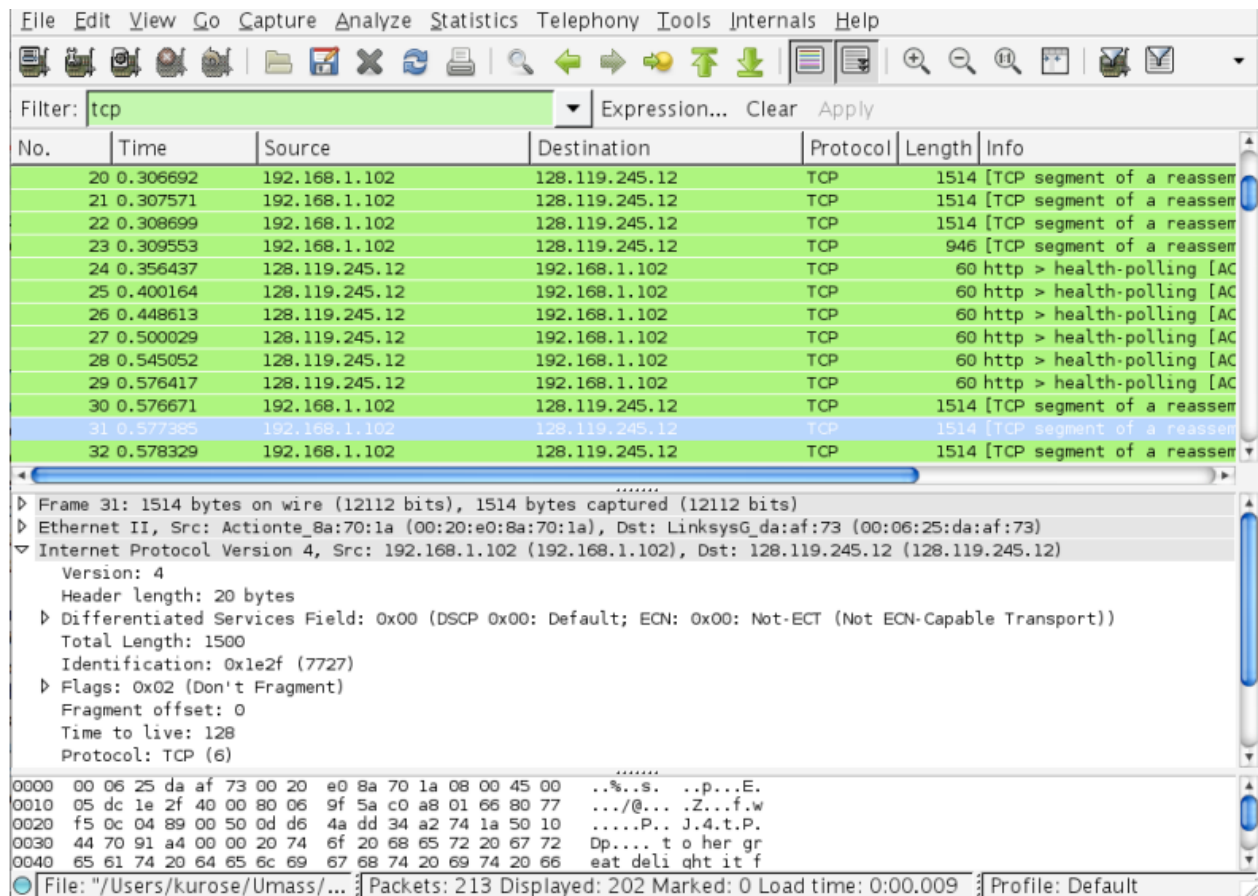


Figure 9: Wireshark capture showing TCP segments for a bulk file transfer.